Agent Analytics MVP

WEBZERO ENGINEERING SPRINT · 5 DAYS

Oct 14, 2026 · @Arunima @Aniruddha M

> **Can we turn observable AI-agent traffic into trustworthy evidence about how agents use enterprise websites, and use that evidence to decide which journeys Webzero should test next?**

In five days, build a working system that sees agent traffic, explains it honestly, and tells us where to point Agentrek next.

At a glance

Mission	Build a working Agent Analytics MVP that observes, classifies and explains AI-agent traffic on a website, then turns it into a ranked list of journeys to test.
Timebox	5 working days, one engineer
Ship	One working end-to-end path: ingest → classify → analyse → dashboard → recommendations
Proving ground	A website Webzero controls, or a representative log dataset
Win condition	At least three evidence-linked test recommendations a Webzero PM would act on
Not required	Perfect bot coverage, many integrations, ML, a polished UI, production certification

01 · The problem

AI agents are a new class of website user, and today's analytics can't see them clearly. Page views, sessions and referrers don't tell a business which agents arrived, what they tried to do, or whether they succeeded. Webzero builds infrastructure for the Agentic

Web, and that starts with measurement.

Agent Analytics must answer five questions, each one building on the last:

#	Question	What the system must establish
1	**Who is visiting?**	AI crawlers, search bots, known agent clients, browser automation and other automated traffic, as far as the evidence supports
2	**What are they doing?**	URLs and resources accessed; interactive elements and workflow steps where observable
3	**What happens?**	Successes, blocks, errors, retries, incomplete journeys and observable outcomes
4	**What does it mean?**	Which pages, capabilities and journeys matter to agents, and where problems show up
5	**What should we test next?**	Evidence-backed recommendations that feed Webzero's journey simulation and optimisation

02 · The evidence rule

Every claim the system makes must be labelled as **observed**, **inferred** or **unknown**, and the UI must show which. This rule outranks every feature below.

Label	Meaning	Example
Observed	Directly recorded fact	`GET /pricing` returned 403 at 10:42:07 to a client claiming to be GPTBot
Inferred	A reasonable hypothesis, with its evidence and confidence	These 14 requests likely form one discovery session (medium confidence)
Unknown	Something the data cannot establish	Whether the agent understood the page or completed the task

Three traps to avoid:

- **A crawler hit is not a journey.** A request from an AI crawler does not prove an agent attempted a customer task.
- **A 200 is not success.** A successful response does not mean the content was understood or the task completed.
- **A User-Agent is a claim, not an identity.** Show "claims to be X, unverified" until the source is independently verified (e.g. reverse DNS or published IP ranges).

03 · What you'll build

Four capabilities feed the recommendation engine in section 04. Build them thin, end to end, and early.

A. Collection and ingestion

Ship **one** documented, reproducible collection path that produces real data. Compare the options before choosing (on coverage, effort, accuracy, privacy, cost and enterprise fit) and record what each can and cannot see.

Source	Sees	Misses
Server / app access logs	Every HTTP request, including non-JS crawlers	Clicks, form steps, task completion
CDN, proxy or WAF logs	Edge traffic, blocks, bot signals	Same as access logs; vendor-specific formats
JS snippet	Navigation and UI actions on instrumented pages	Crawlers and any client that doesn't run JavaScript
Middleware / app hook	Server-side workflow transitions	Anything outside the instrumented app
Structured log import (JSON/CSV)	Whatever the source captured	Real-time data

Stay provider-neutral: Cloudflare can be one integration, never the data model. Integrations you can't finish become documented extensions.

B. Identification and classification

Classify every observation into: verified search crawlers · known AI crawlers · other identifiable bots · browser automation or agent-like activity (when evidence supports it) · ordinary browsers · unknown.

Each normalised event records, where the source allows:

Group	Fields
Request	Timestamp, hostname, method, path, status, response time, referrer, request/session ID
Client	User-Agent, source IP (masked or hashed)
Verdict	Classification, claimed vs verified identity, confidence, evidence, detection method, source, rule version

Rules live in an **extensible registry**: adding or changing an agent signature must not touch pipeline code.

C. Activity and journey reconstruction

Group events into sessions only where evidence permits. Same IP ≠ same agent; close in time ≠ same journey. For each activity group show:

- Start and last-seen times, traffic type, identity confidence and evidence
- Pages in order, request and distinct-page counts
- Status codes, errors and retry patterns
- The observable step sequence and outcome, if one can be established
- The gaps that limit the conclusion

With client-side instrumentation, add navigation, UI actions and workflow transitions. Never capture agent reasoning, credentials or sensitive form contents.

D. Dashboard

Every metric must click through to the evidence behind it.

View	Shows
Overview	Total requests, share that is AI or automated, distinct agents, traffic by class and confidence, top pages, error and denial rates, trends
Agent directory	Name, category, verification status, first and last seen, volume, distinct pages, common paths, links to sessions
Activity explorer	Filterable, searchable event table and timeline; per-event classification evidence; session drill-down
Page and journey insights	Agent-heavy pages, pages with errors, blocks or retries, important destinations agents never reach, workflow attempts and outcomes, coverage limits

04 · The centrepiece: test recommendations

Reporting traffic is the means; the output is a prioritised list of journeys for Webzero to test with Agentrek or another simulation method, not generic optimisation tips. Transparent rules are fine for v1; skip the ML.

Every recommendation carries eight fields:

Field	Answers
Recommended test	Which page, task or journey to test
Reason	The behaviour or failure that triggered it
Evidence	Linked events, URLs, status codes, timestamps, sequences
Potential issue	What may be blocking the agent
Suggested validation	How to reproduce or investigate
Priority	A transparent score (below)
Confidence	How strongly the evidence supports it

Field	Answers
Next action	The concrete test or investigation to run

Priority must be explainable. A starting point to refine:

```
\text{Priority} = \text{Impact} \times \text{Frequency} \times
\text{Severity} \times \text{Confidence}
```

Five patterns the engine should recognise:

Pattern	Trigger in the data	Hypothesis to validate
Product discovery	An AI client repeatedly reads product pages but never reaches application or conversion pages	Pages may lack usable links or clear next steps
Blocked access	Relevant clients repeatedly get 403s on important public pages	Restrictions may be blocking legitimate discovery
Broken navigation	Repeated failures or redirect loops around a key journey	A route is broken; reproduce and trace it
Content discoverability	Key product information never appears in what a crawler fetched	Content may be inaccessible or poorly structured
Full-journey simulation	A critical workflow matters but logs can't show completion	Only a controlled simulation can measure it

A status code or a missing next request is never, on its own, proof of a failed journey. Each recommendation states its evidence, its uncertainty, and why simulation is the way to settle it.

05 · Enterprise guardrails

Design for regulated buyers such as banks from day one. These are design requirements, not a claim that a five-day build will pass compliance.

Area	Requirements
Privacy and security	Never capture passwords, tokens, payment data or form values. Strip query strings that may hold PII or secrets. Mask or hash IPs; configurable retention. Restrict data access. Secrets in env vars or a secrets manager. Validate and sanitise all input. Document data flows, storage, retention and deletion.
Reliability	Collection must never slow or break the site. Ingest asynchronously. Survive malformed and duplicate events. Error logging, health checks, and an alert when data stops arriving.
Deployment	Separate collection, normalisation, classification, analytics and presentation. Provider-neutral event schema; swappable integrations. No needless vendor lock-in or costly infrastructure. Documented config and dependencies.
Auditability	Keep the sanitised evidence behind every classification and recommendation. Record when, how and by which rule each event was classified. Separate facts from inferences. Document false positives and limits.

06 · Research mandate

Research justifies the architecture; it is not the deliverable. Spend Day 1 on it, then build. Before committing, find out:

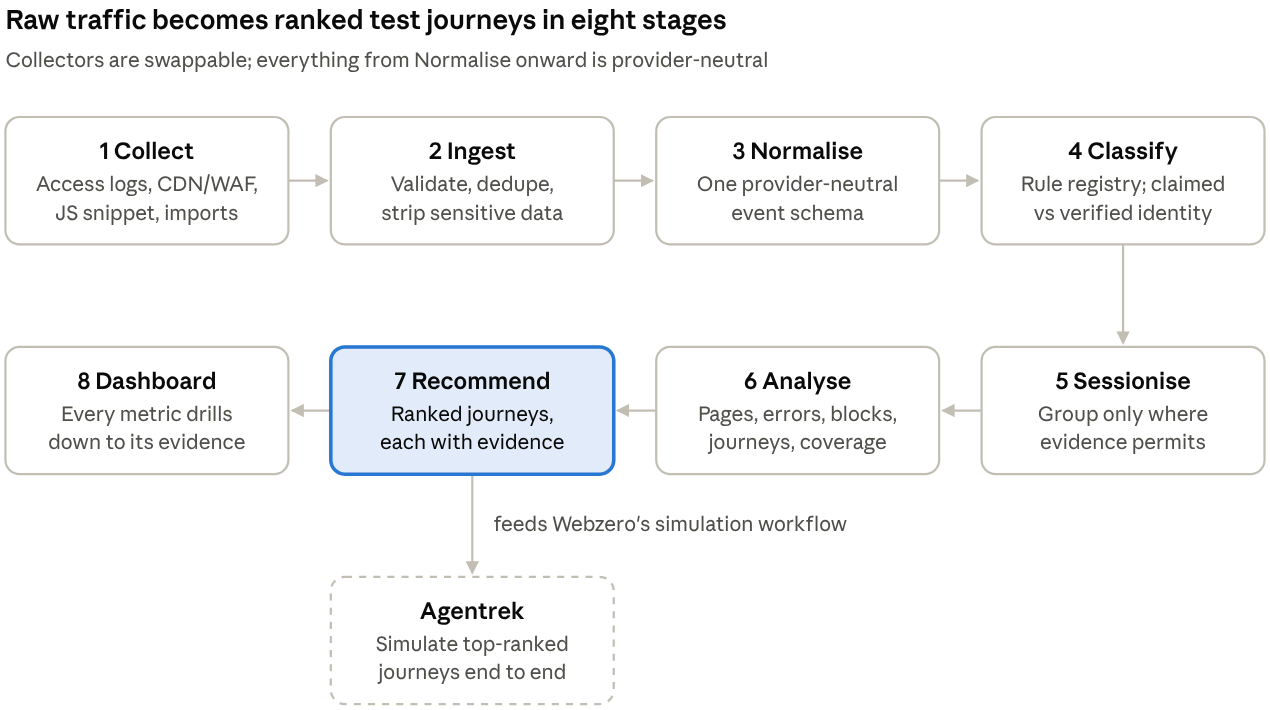
1. How major AI crawlers and documented agent clients identify themselves
2. How each identity can be verified, and where User-Agent detection breaks down
3. What server logs, CDN logs and browser instrumentation can each reveal
4. How to tell search crawlers, AI bots, browser agents and humans apart
5. How to sessionise events without unsupported assumptions
6. Which HTTP, event and observability conventions (e.g. OpenTelemetry) suit an extensible schema

7. Enterprise security, privacy and retention practice for analytics data
8. Open-source tools and libraries that save build time

Prefer official documentation, standards and established security guidance. Log each source with the decision it informed.

07 · Reference architecture

A suggested shape, not a mandate: you choose the stack. Keep it to separable stages, not microservices.



Agent Analytics pipeline · 8 stages

Recommendations are the stage that matters; everything upstream exists to make them trustworthy.

08 · Five days

A thin end-to-end path is live by Day 2; every later day deepens it

	FOCUS	SHIPS
DAY 1 **Research & architecture**	Data sources, bot verification, privacy Event schema, classifier, test fixture	ADR, sources, data model, project skeleton
DAY 2 **Collect & classify**	One ingestion path, parsed + normalised Rules, confidence, spoofed-UA fixtures	**End-to-end path live:** classified observations
DAY 3 **Analytics & explorer**	Sessions, aggregates and filters Dashboard with classification evidence	Usable UI on real events
DAY 4 **Recommend & harden**	Evidence-linked recommendation rules Privacy, access control, edge-case tests	Ranked, evidence-backed test opportunities
DAY 5 **Validate & hand over**	Run on a controlled site; fix defects Limits, failure modes, roadmap	Demo, code, setup guide, test evidence, handover

Five-day plan · focus and daily deliverable

The plan is a guide, not a reason to leave integration until Friday. If an integration won't land, build a log importer or local collector, document the trade-off, and keep moving.

09 · Definition of done

The MVP is done when it is functional, explainable, testable, and useful enough to inform a real Webzero product decision.

- [ ] Ingests real HTTP access logs or equivalent events through a documented, reproducible path
- [ ] Normalises every event into one internal schema
- [ ] Classifies a useful subset of traffic, with evidence and confidence recorded
- [ ] Shows meaningful analytics in a working interface
- [ ] Lets a user inspect any event and its related activity

- [ ] Produces **at least three** evidence-linked test recommendations
- [ ] Visibly separates observed facts, hypotheses and unknowns
- [ ] Has automated tests for the pipeline and classification logic
- [ ] Handles sensitive fields responsibly and documents remaining security gaps
- [ ] Can be run by another engineer from the handover alone

10 · How it will be judged

Criterion	The question we'll ask
Functional completeness	Does the whole flow work, or is it mostly a mock-up?
Technical judgement	Were the right trade-offs made for five days?
Research quality	Are key decisions backed by credible sources and verified behaviour?
Data integrity	Are identities and conclusions accurate, with uncertainty explicit?
Product thinking	Do the insights tell us what to test next?
Enterprise awareness	Were privacy, security, reliability and deployment considered?
Code quality	Is it understandable, testable and extendable?
Ownership	Were risks raised early, progress shared and feedback used?
Handover	Can someone else reproduce it and understand its limits?

11 · Handover package

- [ ] Source code with reproducible setup instructions
- [ ] Architecture decision record and research references with links
- [ ] Event schema and classification rules
- [ ] Working ingestion flow and dashboard, with sample data or a controlled demo setup

- [ ] Test cases and results
- [ ] Evidence-backed test recommendations
- [ ] Known limitations, security considerations and a prioritised path to production

---

The final review. We are not judging polish or completeness. We are looking for sound judgement, and for a system someone can pick up and use. Come ready to run it live and to answer: What did you build, and why this architecture? What does the data prove, and where does it fall silent? Which journeys should Webzero test next? What stands between this and production?

Webzero · Infrastructure for the Agentic Web